



MASTER THESIS

COURSE CODE - COURSE NAME

Biologically Plausible Alternatives To Backpropagation For Training Neural Networks

Author:
Mats Wiig Martinussen

Date

Table of Contents

List of Figures	ii
List of Tables	ii
1 Introduction	1
1.1 Background and Motivation	1
1.1.1 Recent Progress in Artificial Intelligence	1
1.1.2 Open Challenges in Current AI Systems	1
1.1.3 Two Perspectives on the Path Toward General Intelligence	2
1.1.4 Biological Systems as a Reference for General Intelligence	2
1.1.5 Biological Inspiration: Architecture and Learning	3
1.1.6 Motivation for Alternative Learning Rules	3
1.2 Contribution, Research Objectives and Research Questions	3
1.2.1 Contribution	3
1.2.2 Objectives	3
1.2.3 Research Questions	4
1.3 Structure of the Thesis	4
2 Theory	4
2.1 How the brain learns	4
2.1.1 Hebbian Learning	4
2.1.2 Neuromodulators and global feedback	4
2.2 Learning in artificial neural networks	4
2.2.1 Supervised learning	4
2.2.2 Backpropagation	4
2.2.3 Biologically plausible alternatives to backpropagation	4
3 Method	5
3.1 Weight Perturbation	5
3.1.1 Definition and Derivation	5
3.1.2 Induced pre-activation noise	7
3.2 Standard Node perturbation	8
3.3 Induced-Noise Node perturbation	8
3.3.1 INP as a noise-reduced version of WP	9
3.4 Evaluation Framework for INP	10

3.4.1	Directional Alignment: Cosine Similarity	11
3.4.2	Variance of the Gradient Estimator	14
3.4.3	Stability of the Learning Dynamics	16
3.5	Theory-Driven Extensions to INP	19
3.5.1	TO BE REMOVED (most likely): Geometry Aware Perturbation	19
4	Case Study	21
4.1	Experimental Setup	21
4.2	Learning Problems and Datasets	21
4.3	Algorithms	21
4.4	Results	21
5	Concluding Remarks	21
5.1	Discussion	21
5.2	Future Work	21
5.3	Conclusion	21
	Appendix	22

List of Figures

List of Tables

1	Comparison of perturbation noise sampling in the literature.	23
2	Comparison of perturbation-based update rules in the literature.	24

1 Introduction

1.1 Background and Motivation

1.1.1 Recent Progress in Artificial Intelligence

Artificial intelligence has undergone a dramatic transformation over the past decade, emerging as one of the most powerful technological forces of our time. Recent advances in deep learning and large-scale optimization have produced systems capable of performing tasks once considered exclusive to human intelligence. Large language models can generate coherent text, write code, and assist in complex reasoning tasks. Reinforcement learning systems have achieved superhuman performance in games such as chess and Go, while breakthroughs in scientific machine learning have enabled progress on long-standing challenges such as protein structure prediction. At the same time, autonomous systems are transitioning from research prototypes to real-world deployment, with self-driving vehicles now operating in urban environments. Taken together, these developments suggest that artificial intelligence is not only progressing rapidly, but may be approaching a level of capability that raises fundamental questions about the feasibility of artificial general intelligence.

1.1.2 Open Challenges in Current AI Systems

Despite this rapid progress, there is increasing debate within the research community regarding whether current approaches are sufficient to achieve artificial general intelligence. Yann LeCun, often referred to as one of the “godfathers of AI”, has been particularly vocal in his criticism of current paradigms, recently co-founding a company backed by over \$1 billion to pursue fundamentally different approaches to intelligence. These developments suggest that, despite impressive empirical success, fundamental challenges may still remain. Among others, these include:

- **Energy intensity.** The 10^{11} neurons and 10^{15} synapses contained in the typical human brain expend approximately 20 W of power, whereas a digital simulation of an artificial neural network of approximately the same size consumes 7.9 MW. Source: Wong, T. M. et al. 1014. Report no. RJ10502 (ALM1211-004) (IBM, 2012). The power consumption of this simulation of the brain puts that of conventional digital systems into context. You can also include a graph that show the energy consumption per model since 1985. Exponential graph. Cant keep growing like this forever. So maybe right now, the real bottleneck to intelligence is energy. So we need more energy efficient architectures to achieve AGI.
- **Lack of continual learning.** Most modern AI systems lack the ability to learn continuously from experience. During training, models are optimized on a fixed dataset, after which their parameters remain largely static during deployment, preventing them from naturally adapting to new information. While updates through fine-tuning are possible, these often suffer from *catastrophic forgetting*, where newly acquired knowledge interferes with previously learned information. This limitation is problematic because a generally intelligent system should be able to incorporate new information as it becomes available—for example, adapting to a specific user or immediately integrating major real-world events—without degrading prior knowledge. Current approaches such as in-context learning partially address this by conditioning on recent inputs, but this remains a temporary and indirect workaround rather than true persistent learning. As such, the lack of robust continual learning mechanisms represents a key limitation of current AI systems.
- **Limited out-of-distribution generalization.** Current AI systems often perform well on tasks that resemble their training data, but their performance can degrade significantly when faced with novel situations. This is because most models rely on the assumption that training and test data follow similar distributions, an assumption that rarely holds in real-world settings. As a result, models tend to exploit statistical regularities or spurious correlations present in the training data, rather than learning underlying structure that transfers to new

environments. Recent work has shown that even when models appear robust on standard benchmarks, significant failure modes can emerge under distribution shifts, indicating that current evaluation methods may overestimate true generalization performance. Since general intelligence fundamentally requires the ability to apply knowledge to new and unseen situations, limited out-of-distribution generalization represents a central challenge for current AI systems.

- **Limited abstraction.** Current AI systems often struggle to learn high-level, abstract concepts that generalize across different contexts. Instead, they tend to rely on statistical regularities and surface-level features present in the training data, rather than capturing the underlying meaning of a concept. For example, a model trained to recognize chairs may rely on features such as the presence of four legs, and fail to identify unconventional designs that do not match these patterns, despite clearly serving the same purpose. In contrast, the abstract concept of a chair is simply something to sit on, independent of its specific appearance. As a result, models may perform well on familiar inputs but fail to generalize when superficial characteristics change.
- **Lack of grounding in the physical world.** Most modern AI systems learn from static datasets rather than through interaction with the environment, limiting their ability to connect representations to real-world experience. As a result, their knowledge is largely derived from correlations in data rather than direct interaction with the underlying phenomena. For example, a language model may learn about cooking by reading large numbers of recipes, but it has no direct experience of taste, texture, or smell, and therefore lacks a grounded understanding of what the concepts it describes actually correspond to in the real world. This makes their behavior highly dependent on the data they were trained on and limits their ability to reason about concepts that are inherently tied to physical interaction.

1.1.3 Two Perspectives on the Path Toward General Intelligence

There are broadly two perspectives on the trajectory of current AI systems. On the one hand, some researchers argue that existing approaches may be fundamentally limited, lacking key ingredients required for general intelligence. On the other hand, there is a belief that continued scaling of data, model size, and computational resources will lead to increasingly general capabilities, with no clear evidence of saturation so far.

Regardless of which perspective proves correct, systems that achieve general intelligence must exhibit the capabilities outlined above, such as continual learning, strong generalization, and the ability to form abstract and grounded representations. Even if these properties ultimately emerge from scaling alone, it remains unclear whether current approaches represent the most efficient or principled way of achieving them. The natural thing to do is, therefore, to look at systems that already exhibit these capabilities.

1.1.4 Biological Systems as a Reference for General Intelligence

Biological systems provide a proof of existence that the capabilities outlined above can be achieved in practice. Humans and other animals exhibit continual learning, adapting to new experiences without requiring a separate training phase. They demonstrate strong generalization, applying knowledge to novel situations—for example, recognizing objects under new conditions or solving problems they have not explicitly encountered before.

In addition, biological learning is highly sample-efficient. A child can learn the meaning of a new word, such as “butterfly,” from only a few exposures by leveraging contextual cues, such as where a speaker is looking or pointing, rather than requiring repeated observations across large datasets. Biological systems also form abstract representations, allowing them to recognize that an object is a chair regardless of its specific appearance, as long as it serves the function of something to sit on.

Finally, biological intelligence is grounded in interaction with the physical world. Concepts such as “heavy” or “taste” are not learned through descriptions alone, but through direct sensory experience and interaction. Together, these properties highlight that the challenges identified in current AI systems are routinely addressed in biological systems, suggesting that they may provide a valuable source of inspiration for developing more general and adaptive learning mechanisms.

1.1.5 Biological Inspiration: Architecture and Learning

Research inspired by biological systems broadly follows two directions. One approach focuses on developing architectures that more closely resemble biological neural systems. This includes neuromorphic hardware **I could show an example or two here, and generally spend some more time on neuromorphic architectures**, but also more radical approaches where biological neurons themselves are used as the computational substrate. A particularly striking example is the CL1 system developed by Cortical Labs, where networks of living neurons are interfaced with a digital environment and trained to perform tasks through real-time interaction. In such systems, neural activity both drives and responds to external inputs, forming a closed feedback loop that allows the system to adapt its behavior over time. Despite consisting of relatively simple neural cultures, these systems have demonstrated the ability to learn goal-directed behavior, highlighting the potential of biological substrates for information processing and learning. While still in an early stage, this line of work represents a fundamentally different paradigm from conventional AI, where computation is no longer confined to silicon, but emerges from the dynamics of living neural systems.

An alternative approach is to focus not on replicating the full biological architecture, but on understanding and emulating the mechanisms by which such systems learn. Rather than building biological systems directly, this direction seeks to identify learning rules that capture key properties of biological learning. In this thesis, we focus on this second direction, and in particular on biologically plausible learning algorithms.

1.1.6 Motivation for Alternative Learning Rules

Backpropagation is not biologically plausible. It relies on mechanisms such as exact gradient computation, symmetric weight transport, and the propagation of global error signals through the network—none of which are known to be implementable in biological neural systems. If architectures that more closely resemble biological systems, such as CL1, turn out to be a viable path toward more general intelligence, then backpropagation is no longer applicable as a learning mechanism. In such settings, alternative learning rules are not just interesting, but necessary.

At the same time, even within current machine learning systems, backpropagation is not without limitations. It is computationally and memory intensive, and its effectiveness relies on highly structured training procedures and large amounts of data. This raises the question of whether alternative learning mechanisms, potentially inspired by biological systems, can offer different trade-offs or point toward new directions for improving existing approaches.

1.2 Contribution, Research Objectives and Research Questions

1.2.1 Contribution

1.2.2 Objectives

The objective of this thesis is to investigate perturbation-based learning algorithms, with a particular focus on node and weight perturbation, as biologically plausible alternatives to backpropagation.

- **Evaluate performance and learning dynamics.** Assess how perturbation-based learning methods perform across a range of tasks, both empirically and theoretically. This includes

analyzing convergence behavior, stability, and efficiency, and establishing metrics that allow for meaningful comparison with backpropagation.

- **Explore improvements within perturbation-based methods.** Investigate modifications and extensions to node and weight perturbation aimed at improving their performance, robustness, and scalability.
- **Extract insights for alternative learning mechanisms.** Examine whether the behavior and properties of perturbation-based learning can provide insights into alternative optimization strategies, potentially informing the development of improved or complementary approaches to backpropagation.

1.2.3 Research Questions

1.3 Structure of the Thesis

2 Theory

2.1 How the brain learns

2.1.1 Hebbian Learning

2.1.2 Neuromodulators and global feedback

2.2 Learning in artificial neural networks

2.2.1 Supervised learning

2.2.2 Backpropagation

2.2.3 Biologically plausible alternatives to backpropagation

Move the method section of weight perturbation and node perturbation here. Give FA and FF (or some other methods) a relatively short and compact introduction, explain how they work, why they are biologically plausible, and how well they work. Do the same for perturbation methods, then conclude that in this thesis, we are going to focus on perturbation methods. Then introduce wp and standard np. Then the noise-induced version of np will be presented in the theory section.

- Feedback Alignment
- Forward Forward
- STDP
- Perturbation methods

Table 1 compares the learning rules used in different perturbation-based methods. The table shows where noise is added, how the noise is sampled, and how the weights are updated. We use σ for the noise size in all papers, even when the original paper uses another symbol. We use $\Delta\ell$ to denote the scalar global learning signal, typically computed from the difference between noisy and unperturbed performance, although the exact estimator varies between papers.

For compactness, let U_k denote the update direction, so that weights are updated as $W_k \leftarrow W_k - \eta_W U_k$. For decorrelated variants, $x_{k-1}^* = R_{k-1} x_{k-1}$, and the decorrelation matrix is updated using $R_k \leftarrow R_k - \eta_R (x_k^* (x_k^*)^\top - \text{diag}((x_k^*)^2)) R_k$.

3 Method

3.1 Weight Perturbation

3.1.1 Definition and Derivation

Perturbation-based learning provides a simple alternative to backpropagation by estimating gradients using only forward evaluations of the network. Instead of explicitly computing derivatives, the idea is to introduce small random perturbations to the parameters and observe how these perturbations affect the loss. If a perturbation leads to a decrease in loss, then moving the parameters in that direction is beneficial; if it increases the loss, the direction is unfavorable. By correlating random perturbations with the resulting change in loss, one can construct a stochastic estimate of the gradient.

To formalize this perturbation-based approach, we model the network parameters as noisy rather than fixed. Let $\boldsymbol{\mu}$ denote the deterministic parameter vector that we seek to optimize, and define perturbed parameters by

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I}),$$

where $\sigma > 0$ is a small fixed perturbation scale. Rather than optimizing the deterministic loss $L(\boldsymbol{\mu})$ directly, we consider the expected loss under parameter perturbations,

$$F(\boldsymbol{\mu}) := \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta})], \quad (1)$$

where $p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$. We now seek to construct an estimator of the gradient of this objective. As we will show, it is possible to obtain an unbiased estimate of $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$ using only forward evaluations of the network. For sufficiently small σ , this gradient closely approximates the gradient of the original loss $L(\boldsymbol{\mu})$.

To compute $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$, we write the expectation explicitly as

$$F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta}) p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) d\boldsymbol{\theta},$$

and differentiate under the integral sign:

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) d\boldsymbol{\theta}.$$

This step is valid under standard regularity conditions, and allows us to make the dependence on $\boldsymbol{\mu}$ explicit inside the integral. Following the score-function estimator [Williams1992], we use the identity

$$\nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}),$$

which yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})]. \quad (2)$$

This expression rewrites the gradient of an expectation as an expectation involving the score function $\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})$, enabling gradient estimation using Monte Carlo sampling.

For the Gaussian distribution $p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$, the log-density is given by

$$\log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = -\frac{1}{2\sigma^2} \|\boldsymbol{\theta} - \boldsymbol{\mu}\|^2 + C,$$

and its gradient is

$$\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2}.$$

Substituting into (2) yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\theta}) \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2} \right].$$

Using the reparameterization $\boldsymbol{\theta} = \boldsymbol{\mu} + \sigma \boldsymbol{\xi}$, we obtain

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\boldsymbol{\xi}}{\sigma} \right]. \quad (3)$$

In practice, the expectation in (3) cannot be computed exactly, as it requires integrating over all possible perturbations. Instead, we approximate it using Monte Carlo sampling, which allows us to estimate the gradient using only forward evaluations of the network. For a single sample $\boldsymbol{\xi}$, this yields the estimator

$$\hat{g}_i = L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\xi_i}{\sigma}.$$

To reduce the variance of this estimator, we subtract a baseline b that does not depend on the individual perturbation ξ_i . The estimator remains unbiased, since

$$\mathbb{E} \left[b \frac{\xi_i}{\sigma} \right] = 0,$$

as ξ_i has zero mean and b is independent of ξ_i . In particular, we use a leave-one-out baseline, where for the k -th perturbation we define

$$b^{(k)} = \frac{1}{N-1} \sum_{k' \neq k} L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}^{(k')}).$$

Intuitively, the baseline centers the learning signal: perturbations that lead to lower-than-average loss reinforce their contribution, while those that perform worse than average are reversed. This reduces variance in the gradient estimate without affecting its expectation. The resulting estimator becomes

$$\hat{g}_i = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_i}{\sigma}. \quad (4)$$

Applying gradient descent, we update the parameters in the negative gradient direction:

$$\Delta \mu_i = -\eta (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_i}{\sigma}, \quad (5)$$

where η is the learning rate. This corresponds to the weight perturbation update rule. This update rule constitutes an unbiased Monte Carlo estimator of the gradient of the expected loss, implying that its expected update direction coincides with the true backpropagation gradient, although potentially with high variance.

A third contribution concerns the choice of baseline in perturbation-based learning. In the literature considered here, baseline subtraction is typically implemented using an additional forward pass. Standard WP/NP methods often subtract the loss from a noiseless forward pass [werfel2005learning, hiratani2022stability, dalm2024effective, zuge2023weight], while noisy-baseline variants subtract the loss from a second independently perturbed forward pass [cho2011node, hara2017statistical]. Both approaches require evaluating an additional baseline trajectory. However, REINFORCE-style estimators remain unbiased when subtracting any baseline that is independent of the current perturbation [Williams1992]. We therefore replace the extra baseline forward pass with a leave-one-out batch baseline. For the k -th perturbation sample, we define

$$b^{(k)} = \frac{1}{K-1} \sum_{m \neq k} L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}^{(m)}),$$

which is independent of $\boldsymbol{\xi}^{(k)}$. Hence,

$$\mathbb{E} \left[b^{(k)} \frac{\xi_i^{(k)}}{\sigma} \right] = 0,$$

so the estimator remains unbiased. Intuitively, this baseline centers the learning signal using the average performance of the other perturbations in the same batch: perturbations with lower-than-average loss are reinforced, while perturbations with higher-than-average loss are reversed. When the perturbation batch is large, this leave-one-out mean should provide a reasonable estimate of the expected perturbed loss, and may behave similarly to a separate baseline evaluation while avoiding its additional forward-pass cost. This makes the method computationally attractive, especially when many perturbations are already evaluated in parallel. However, this argument does not prove that the leave-one-out baseline is always better than a noiseless or independently perturbed baseline. Its practical usefulness therefore needs to be validated empirically, by comparing estimator variance, cosine similarity, and training performance against the standard baseline choices.

3.1.2 Induced pre-activation noise

To understand how the weight perturbation update affects the network, we examine how perturbations at the level of the weights induce noise in the pre-activations. Recall that in the previous section, we modeled the parameters as $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$. For neural networks, this corresponds to perturbing each weight independently as

$$\mathbf{W}^{\ell'} = \mathbf{W}^{\ell} + \sigma \mathbf{E}^{\ell}, \quad E_{ij}^{\ell} \sim \mathcal{N}(0, 1) \text{ i.i.d.}$$

The pre-activation at neuron j in layer ℓ is given by

$$z_j^{\ell} = \sum_i W_{ij}^{\ell} x_i^{\ell-1}.$$

After perturbation, this becomes

$$z_j^{\ell'} = \sum_i W_{ij}^{\ell'} x_i^{\ell-1'} = \sum_i (W_{ij}^{\ell} + \sigma E_{ij}^{\ell}) x_i^{\ell-1'}$$

This expression separates into a deterministic component and a stochastic component induced by the weight perturbations:

$$z_j^{\ell'} = \sum_i W_{ij}^{\ell} x_i^{\ell-1'} + \sigma \sum_i E_{ij}^{\ell} x_i^{\ell-1'}.$$

Define the induced pre-activation noise as

$$n_j^{\ell} := \sigma \sum_i E_{ij}^{\ell} x_i^{\ell-1'}. \tag{6}$$

We can now write the full pre-activation in compact form as

$$z_j^{\ell'} = (\mathbf{W}^{\ell} \mathbf{x}^{\ell-1'})_j + n_j^{\ell},$$

where

$$(\mathbf{W}^{\ell} \mathbf{x}^{\ell-1'})_j = \sum_i W_{ij}^{\ell} x_i^{\ell-1'}.$$

Now condition on the incoming perturbed activations $\mathbf{x}^{\ell-1'}$. Under this conditioning, the coefficients $x_i^{\ell-1'}$ are fixed, while E_{ij}^{ℓ} remain independent standard Gaussians. From (6), it follows that n_j^{ℓ} is a linear combination of independent Gaussian variables, and therefore Gaussian with

$$n_j^{\ell} \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}\left(0, \sigma^2 \sum_i (x_i^{\ell-1'})^2\right) = \mathcal{N}(0, \sigma^2 \|\mathbf{x}^{\ell-1'}\|^2). \tag{7}$$

Thus, weight perturbation induces Gaussian noise at the pre-activations, with variance proportional to the squared norm of the incoming activations. In other words, the effect of perturbing the weights can be fully characterized as injecting structured Gaussian noise directly into the pre-activations.

3.2 Standard Node perturbation

Werfel et al. [werfel2005learning] analytically show, for a single-layer linear perceptron, that node perturbation can achieve a faster optimal learning curve than weight perturbation, with the difference explained by the lower dimensionality of the perturbation space: WP explores the MN -dimensional weight space, whereas NP explores the M -dimensional output space. This dimensionality intuition is often used to motivate NP more generally, but the original result is specific to a linear teacher-student setting. Moreover, the comparison is not fully settled: Züge et al. [zuege2023weight] show that in temporally extended, low-dimensional tasks, WP can perform similarly to or better than NP because WP-induced output perturbations are constrained to the realizable output subspace, while standard NP injects arbitrary time-dependent node noise. This motivates a more direct variance-based comparison between WP and NP in standard feed-forward MLPs. Closely related results exist in other settings: Ren et al. [ren2023scaling] show that activity-perturbed forward-gradient estimators have lower variance than weight-perturbed forward-gradient estimators, and the local reparameterization trick of Kingma et al. [Kingma2015] shows that sampling induced activation noise instead of weight noise can reduce stochastic gradient variance. Inspired by this perspective, we derive the pre-activation noise distribution induced by Gaussian weight perturbations and define an induced-noise node perturbation estimator. We then show that this estimator is the conditional expectation of the corresponding WP estimator given the induced pre-activation noise. By the law of total variance, the induced-noise NP estimator therefore has variance no larger than WP, and strictly lower variance whenever the conditional variance of the WP estimator is nonzero.

3.3 Induced-Noise Node perturbation

Instead of introducing noise at the level of the weights, a more common approach is to inject perturbations directly at the pre-activations. In its simplest form, node perturbation considers

$$z_j^{\ell'} = z_j^\ell + \epsilon_j^\ell, \quad \epsilon_j^\ell \sim \mathcal{N}(0, \sigma^2),$$

and constructs a gradient estimate based on the resulting change in loss [hiratani2022stability].

However, since the goal is to estimate gradients with respect to the weights, the choice of perturbation distribution at the pre-activations is not arbitrary. In particular, it should be consistent with the perturbations induced by weight perturbation, which defines the underlying smoothed objective. As shown in the previous section, Gaussian weight perturbations induce structured Gaussian noise at the pre-activations, with variance depending on the norm of the incoming activations. We therefore define node perturbation using this induced distribution, ensuring that the resulting estimator coincides with weight perturbation in expectation.

$$z_j^{\ell'} = W^\ell x^{\ell-1'} + n_j^\ell, \quad n_j^\ell \mid x^{\ell-1'} \sim \mathcal{N}(0, \sigma^2 \|x^{\ell-1'}\|^2).$$

We first consider the effect of this perturbation at the level of the pre-activations. The resulting gradient estimate with respect to z_j^ℓ is given by

$$\hat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{n_j^\ell}{\sigma^2 \|x^{\ell-1'}\|^2}.$$

Using the chain rule,

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1'},$$

we obtain the corresponding estimator for the weights:

$$\hat{g}_{ij,\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{n_j^\ell x_i^{\ell-1'}}{\sigma^2 \|x^{\ell-1'}\|^2}. \quad (8)$$

Applying gradient descent, the update rule becomes

$$\Delta W_{ij}^\ell = -\eta (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{n_j^\ell x_i^{\ell-1'}}{\sigma^2 \|x^{\ell-1'}\|^2}. \quad (9)$$

Intuitively, node perturbation injects noise at the pre-activations with variance scaled by the magnitude of the incoming activations, ensuring that the resulting perturbations match those induced by weight perturbation. The resulting learning signal is then distributed across the incoming weights proportionally to their contribution to the pre-activation, such that larger inputs receive a larger share of the update.

As in the case of weight perturbation, this estimator is unbiased for the gradient of the same smoothed objective, and the two methods coincide in expectation. This raises a natural question: if the estimators are equivalent in expectation, how do they differ in terms of variance?

3.3.1 INP as a noise-reduced version of WP

We now analyze the variance of the two estimators. A common intuition in the literature is that node perturbation should exhibit lower variance than weight perturbation, as it introduces fewer stochastic degrees of freedom by perturbing at the level of the nodes rather than the individual weights. In the setting considered here, where node perturbation is defined to match weight perturbation in expectation, this intuition can be made precise. In particular, we show that the node perturbation estimator arises as the conditional expectation of the weight perturbation estimator, yielding a variance reduction via the law of total variance.

Proof. We consider the weight perturbation estimator in (4), and analyze its variance by conditioning on the induced pre-activation noise. Recall from (6) that n_j^ℓ is a linear combination of the weight perturbations.

Condition on the full induced noise field $N = (n^1, \dots, n^L)$. Since the perturbed forward pass satisfies

$$z^{\ell'} = W^\ell x^{\ell-1'} + n^\ell,$$

the entire perturbed trajectory $(x^{1'}, \dots, x^{L'})$ is uniquely determined by N . Hence both the learning signal $\delta L(N)$ and all activations $x^{\ell'}$ are deterministic given N .

Now fix (i, j, ℓ) . From (6), and since E_{ij}^ℓ is Gaussian, standard Gaussian regression gives

$$\mathbb{E}[E_{ij}^\ell \mid N] = \frac{x_i^{\ell-1'}}{\sigma \|x^{\ell-1'}\|^2} n_j^\ell.$$

Therefore,

$$\mathbb{E}[g_{ij,\ell}^{\text{WP}} \mid N] = -\frac{\delta L(N)}{\sigma} \mathbb{E}[E_{ij}^\ell \mid N] = -\delta L(N) \frac{n_j^\ell x_i^{\ell-1'}}{\sigma^2 \|x^{\ell-1'}\|^2} = g_{ij,\ell}^{\text{NP}}.$$

Applying the law of total variance,

$$\text{Var}(g_{ij,\ell}^{\text{WP}}) = \text{Var}(\mathbb{E}[g_{ij,\ell}^{\text{WP}} \mid N]) + \mathbb{E}[\text{Var}(g_{ij,\ell}^{\text{WP}} \mid N)].$$

Using the identity above,

$$\text{Var}(g_{ij,\ell}^{\text{WP}}) = \text{Var}(g_{ij,\ell}^{\text{NP}}) + \mathbb{E}[\text{Var}(g_{ij,\ell}^{\text{WP}} \mid N)].$$

Since the second term is nonnegative, it follows that

$$\text{Var}(g_{ij,\ell}^{\text{NP}}) \leq \text{Var}(g_{ij,\ell}^{\text{WP}}). \quad \square$$

This can be understood intuitively by considering the effect of conditioning on the pre-activation noise. Once the pre-activation noise n_j^ℓ is fixed, the entire forward pass, and thus the learning signal $\delta L(N)$, is fixed, making the NP update fully determined. In contrast, many different weight-noise realizations $E_{\cdot j}^\ell$ can produce the same n_j^ℓ (as long as more than one input connects to the neuron), so even when the effect on the network is identical, the WP update can still vary. This additional variability does not influence the loss and therefore contributes purely to the variance of the estimator. A similar principle appears in the local reparameterization trick used in variational inference and dropout [Kingma2015], where sampling at the level of activations reduces variance by eliminating redundant sources of randomness.

Since node perturbation can be viewed as a variance-reduced version of weight perturbation, while both estimators coincide in expectation, we focus on node perturbation in the remainder of this thesis, where we develop a detailed analytical framework and propose theoretically motivated improvements. However, we retain weight perturbation as a baseline in the empirical evaluation to validate the theoretical variance reduction derived here.

The induced-noise perspective also motivates the specific sampling rule used in INP. In most of the perturbation-learning literature considered here, node perturbations are sampled from a fixed distribution whose variance is treated as a global or layerwise hyperparameter. This includes standard Gaussian pre-activation noise in recent NP work [hiratani2022stability,dalm2024effective], Gaussian output-node noise in noisy-baseline NP [cho2011node,hara2017statistical], and related fixed-scale perturbation rules based on random-polarity or uncorrelated weight noise [flower1992summed,cauwenberghs1993fa]. Some works do adjust the perturbation scale to match the effect of another perturbation scheme. For example, Werfel et al. [werfel2005learning] scale the standard deviation of the node noise by $\sqrt{N}$, equivalently the variance by N , so that output-node perturbations match the average output noise induced by weight perturbations. Similarly, Züge et al. [zue2023weight] scale the WP variance by the effective input dimension in order to match the effective output perturbation strength of NP. These scalings are important, but they are still global or expectation-level corrections. In contrast, INP uses the conditional pre-activation noise distribution induced by Gaussian WP, derived in Eq. (7). Thus, for each layer and input example, the perturbation variance is scaled by the actual squared norm of the incoming activations, rather than by a fixed hyperparameter or by a coarse fan-in factor. The motivation is that the gradient is ultimately taken with respect to the weights, and fixed-variance NP ignores how strongly a given weight perturbation would affect the corresponding pre-activation. When the incoming activations are small, fixed-variance NP may perturb the node much more strongly than WP would have done; when the incoming activations are large, it may perturb it too weakly. INP therefore matches the conditional effect of WP at the pre-activation level, rather than only matching its average scale under simplifying assumptions. This does not by itself prove that INP must always train better than fixed-variance NP, but it gives a principled reason to expect a better-scaled estimator. The empirical section therefore evaluates whether this induced-noise sampling rule improves estimator quality in practice, primarily through cosine similarity and estimator variance, and secondarily through training and test loss.

3.4 Evaluation Framework for INP

The ultimate objective of any learning algorithm is to achieve low training and test loss, as these quantities determine whether a model successfully learns the task and generalizes to unseen data. However, loss curves alone provide limited diagnostic information. When a perturbation-based learning rule performs poorly, the loss indicates that learning failed, but it does not reveal why the algorithm failed. Conversely, when performance improves, the loss does not clarify which aspect of the learning rule caused the improvement.

To address this limitation, it is useful to introduce additional diagnostic metrics that capture specific properties of the learning dynamics. Such metrics serve two main purposes:

1. **Identification of failure modes.** Diagnostic metrics make it possible to isolate specific mechanisms that degrade learning performance, such as poor directional alignment between the estimated and true gradient, excessive estimator variance, or unstable parameter dynamics.

-
2. **Guidance for algorithm design.** When analytical expressions for these metrics can be derived, they provide targeted objectives for improving learning rules. In particular, terms that negatively affect a metric can often be traced to specific components of the algorithm, suggesting concrete modifications that may improve performance.

In practice, deriving such expressions typically requires simplifying assumptions about the model or data distribution, but they can nevertheless provide valuable insight into the mechanisms that govern learning behaviour.

Since perturbation-based learning relies on stochastic gradient estimators, learning performance is largely determined by the statistical properties of these estimators. Prior theoretical work, in particular the analysis presented in [hiratani2022stability], highlights three quantities that play a central role in determining learning behaviour. These are (i) the directional alignment between the estimated and true gradient, (ii) the variance of the gradient estimator, and (iii) the stability of the resulting parameter dynamics. Each of these metrics captures a different aspect of estimator quality: directional alignment determines whether updates point in a useful direction, estimator variance determines how noisy the updates are, and stability determines whether the learning dynamics remain well-behaved over time. In the following sections, we first discuss the intuition behind these metrics, then analyze how they influence the expected change in the loss, before deriving analytical expressions for them and evaluating them empirically for both node perturbation and weight perturbation on sinus regression and California housing price regression.

3.4.1 Directional Alignment: Cosine Similarity

Directional alignment measures how well the estimated gradient produced by a perturbation-based learning rule points in the same direction as the true gradient of the loss. Intuitively, if the update direction is well aligned with the true gradient, the learning rule is likely to reduce the loss efficiently. Conversely, if the estimated gradient frequently points in incorrect directions, learning becomes inefficient or may even increase the loss. A natural way to quantify this alignment is through the cosine similarity between the estimated gradient $\hat{g}$ and the true gradient g :

$$\cos(\hat{g}, g) = \frac{\hat{g}^\top g}{\|\hat{g}\| \|g\|} \quad (10)$$

Here, $\hat{g}$ denotes the gradient estimator produced by the learning rule, while $g = \nabla L(\theta)$ represents the true gradient of the loss with respect to the parameters. The numerator measures the inner product between the two vectors, while the denominator normalizes by their magnitudes. As a result, the metric takes values in the interval $[-1, 1]$, where 1 indicates perfect alignment, 0 indicates orthogonality, and negative values indicate that the update direction tends to increase the loss. In the following section, we analyze how this quantity influences the expected change in the loss.

First-order relation between cosine similarity and loss change.

The role of directional alignment can be made precise by examining how a parameter update affects the loss. Let $L(\theta)$ be a differentiable loss function and consider a single update step of the form

$$\theta^+ = \theta + \Delta\theta, \quad \Delta\theta = -\eta \hat{g},$$

where $\eta > 0$ is the learning rate and $\hat{g}$ denotes a stochastic gradient estimator. A first-order Taylor expansion of the loss around θ gives

$$L(\theta^+) = L(\theta + \Delta\theta) \approx L(\theta) + \nabla_\theta L(\theta)^\top \Delta\theta, \quad (11)$$

where higher-order terms in $\|\Delta\theta\|$ are neglected.

Writing the true gradient as $g = \nabla_\theta L(\theta)$ and substituting $\Delta\theta = -\eta \hat{g}$ yields

$$\Delta L = L(\theta^+) - L(\theta) \approx -\eta g^\top \hat{g}. \quad (12)$$

The inner product between the true gradient and the estimated gradient can be decomposed into their norms and the cosine similarity:

$$g^\top \hat{g} = \|g\| \|\hat{g}\| \cos(\hat{g}, g), \quad (13)$$

which gives

$$\Delta L \approx -\eta \|g\| \|\hat{g}\| \cos(\hat{g}, g). \quad (14)$$

Equation (14) shows that, in the first-order regime where the Taylor approximation (11) is valid, the change in loss is directly modulated by the cosine similarity between the estimated gradient and the true gradient. If $\cos(\hat{g}, g) > 0$, the update is locally descending; if $\cos(\hat{g}, g) = 0$, the update is orthogonal to the true gradient and produces no first-order improvement; and if $\cos(\hat{g}, g) < 0$, the update performs local ascent.

Thus, cosine similarity provides a scale-invariant measure of the *directional quality* of a gradient estimator: for fixed norms $\|g\|$ and $\|\hat{g}\|$, larger cosine similarity implies a larger first-order decrease in the loss. This argument, however, only captures the first-order behavior of the update and does not account for estimator variance or higher-order curvature effects, which will be addressed separately in later sections.

Analytical expression for directional alignment under node perturbation

The goal of this section is to derive an analytical expression for the directional alignment between the node perturbation update and the true gradient. Such an expression allows the cosine similarity to be decomposed into terms that depend on properties of the network, perturbations, and gradients. This decomposition is useful because it reveals which factors influence directional alignment and therefore provides insight into how algorithmic modifications might improve learning performance.

Directional alignment is quantified using the cosine similarity between the node perturbation update and the corresponding stochastic gradient descent (SGD) update. Using the Frobenius inner product for matrices, the cosine similarity can be written as

$$\cos(\delta W_k^{\text{NP}}, \delta W_k^{\text{SGD}}) = \frac{\text{tr}[(\delta W_k^{\text{SGD}})^\top \delta W_k^{\text{NP}}]}{\|\delta W_k^{\text{SGD}}\|_F \|\delta W_k^{\text{NP}}\|_F}. \quad (15)$$

To evaluate this quantity analytically, we first derive an expression for the node perturbation update in the small-perturbation regime.

The node perturbation update is defined as

$$\delta W_k^{\text{NP}} = -\frac{\eta}{\sigma} (\ell(\tilde{x}_K, x_0) - \ell(x_K, x_0)) \xi_k x_{k-1}^\top, \quad (16)$$

where ξ_k denotes the perturbation applied to the pre-activation of layer k , η is the learning rate, and $\tilde{x}_K$ represents the perturbed network output.

Following the analysis of [hiratani2022stability], we consider the limit of small perturbation variance $\sigma \rightarrow 0$. In this regime the network output can be linearized using a first-order Taylor expansion,

$$\tilde{x}_K = x_K + \sigma \sum_{l=1}^K \frac{\partial x_K}{\partial h_l} \xi_l. \quad (17)$$

Substituting this expansion into the loss difference and linearizing the loss yields

$$\ell(\tilde{x}_K, x_0) - \ell(x_K, x_0) \approx \sigma \sum_{l=1}^K \xi_l^\top g_l, \quad (18)$$

where

$$g_k = \frac{\partial \ell}{\partial x_K} \frac{\partial x_K}{\partial h_k} \quad (19)$$

denotes the backpropagated gradient with respect to the pre-activation h_k .

Substituting this result into the node perturbation update gives

$$\delta W_k^{\text{NP}} = -\eta \sum_{l=1}^K \xi_k \xi_l^\top g_l x_{k-1}^\top. \quad (20)$$

For comparison, the corresponding SGD update is

$$\delta W_k^{\text{SGD}} = -\eta g_k x_{k-1}^\top. \quad (21)$$

Using the updates above, Hiratani et al. derive an analytical expression for the expected cosine similarity between the node perturbation and SGD updates. The derivation proceeds by computing the expectations of the three quantities appearing in the cosine similarity:

1. the inner product $\text{tr}[(\delta W_k^{\text{SGD}})^\top \delta W_k^{\text{NP}}]$,
2. the norm of the SGD update $\|\delta W_k^{\text{SGD}}\|_F$,
3. the norm of the node perturbation update $\|\delta W_k^{\text{NP}}\|_F$.

Combining these terms yields the approximation

$$\left\langle \frac{\text{tr}[(\delta W_k^{\text{SGD}})^\top \delta W_k^{\text{NP}}]}{\|\delta W_k^{\text{SGD}}\|_F \|\delta W_k^{\text{NP}}\|_F} \right\rangle \approx \sqrt{\frac{\langle \|g_k\|^2 \|x_{k-1}\|^2 \rangle}{\sum_{l=1}^K (L_k + 2\delta_{kl}) \langle \|g_l\|^2 \|x_{k-1}\|^2 \rangle}}. \quad (22)$$

The quantities appearing in Eq. (22) are defined as follows. The vector g_k denotes the backpropagated gradient with respect to the pre-activation of layer k , while x_{k-1} denotes the activity of the previous layer. The norm $\|\cdot\|$ represents the Euclidean norm. The operator $\langle \cdot \rangle$ denotes expectation taken over both the perturbation noise and the data distribution. The term L_k denotes the width of layer k , i.e., the number of neurons in that layer, and δ_{kl} denotes the Kronecker delta, which is equal to 1 when $k = l$ and 0 otherwise.

Two structural features of Eq. (22) are particularly noteworthy. First, the layer width L_k appears explicitly in the denominator of the expression. Since this term multiplies the sum over layers, increasing the number of neurons in the perturbed layer increases the magnitude of the denominator and therefore reduces the resulting cosine similarity. Second, the numerator contains the expected squared gradient term associated with layer k , while the denominator aggregates the corresponding terms across all layers. Consequently, directional alignment depends on how large the gradient contribution of layer k is relative to the combined contributions of all layers. As the number of layers increases, the denominator includes contributions from a larger set of gradient terms, which reduces the relative contribution of any single layer. If gradient magnitudes are unevenly distributed across layers, this effect can become even more pronounced: layers with small gradients contribute little to the numerator while the denominator still aggregates contributions from all layers, leading to particularly weak cosine similarity for those layers. Taken together, these observations indicate that both increasing layer width and increasing network depth tend to reduce cosine similarity under the current learning rule, suggesting that node perturbation may struggle to maintain good directional alignment as networks scale.

The analytical expression in Eq. (22) relies on two approximations introduced in the derivation of [hiratani2022stability]. First, the analysis assumes the small-perturbation regime $\sigma \rightarrow 0$. In this

regime the network output and the loss can be linearized using first-order Taylor expansions, which allows the loss difference to be expressed in terms of the backpropagated gradients g_k . Under this assumption the node perturbation update in Eq. (20) represents the first-order approximation of the true update rule. Second, the derivation approximates the expectation of the cosine similarity in Eq. (22). The left-hand side of this equation contains the expectation of a ratio, while the right-hand side is obtained by computing the expectations of the numerator and denominator terms separately and then combining them. In other words, the expectation of the ratio is approximated using the ratio of the expected terms that appear in the numerator and denominator. This approximation is used in [hiratani2022stability] to obtain a tractable analytical expression for the cosine similarity. Although this step is not exact in general, it is commonly used in high-dimensional analyses where the norms of random vectors tend to concentrate around their expected values. In such settings the fluctuations of the denominator become relatively small compared to its mean, making the approximation sufficiently accurate for analyzing the qualitative scaling behavior of the learning rule.

3.4.2 Variance of the Gradient Estimator

While cosine similarity captures the directional quality of a gradient estimator, it does not describe how noisy the estimator is. In perturbation-based learning the gradient is obtained through stochastic perturbations, which introduces randomness into the update direction. Even if the estimator is unbiased and well aligned with the true gradient on average, large fluctuations between updates can still lead to inefficient learning and unstable optimization. To quantify this effect we analyze the variance of the gradient estimator.

For a vector-valued estimator such as a gradient, variance is naturally described by the covariance matrix

$$\Sigma = \mathbb{E}[(\hat{g} - \mathbb{E}[\hat{g}])(\hat{g} - \mathbb{E}[\hat{g}])^\top]. \quad (23)$$

Since node perturbation provides an unbiased estimate of the true gradient, $\mathbb{E}[\hat{g}] = g$, this can equivalently be written as

$$\Sigma = \mathbb{E}[(\hat{g} - g)(\hat{g} - g)^\top]. \quad (24)$$

Here $\hat{g}$ denotes the stochastic gradient estimator and $g = \nabla L(\theta)$ is the true gradient of the loss. The matrix Σ therefore captures the variability of the estimator around the true gradient. Its diagonal elements correspond to the variance of individual gradient components, while the off-diagonal terms describe correlations between fluctuations in different parameters. In practice the expectation in this expression cannot be computed exactly and is therefore estimated empirically by sampling multiple realizations of the stochastic gradient estimator. In the following section we analyze how this covariance structure influences the expected change in the loss.

Second-order relation between variance and loss change

In the previous section we analyzed the change in loss using a first-order Taylor expansion, which revealed how directional alignment influences learning. However, this analysis ignores both curvature of the loss landscape and fluctuations in the gradient estimator. To capture these effects we consider a second-order Taylor expansion of the loss.

Let $L(\theta)$ be a twice differentiable loss function and consider the update

$$\theta^+ = \theta + \Delta\theta, \quad \Delta\theta = -\eta \hat{g},$$

where $\hat{g}$ is a stochastic gradient estimator and $\eta > 0$ is the learning rate. A second-order Taylor expansion of the loss around θ gives

$$L(\theta^+) = L(\theta + \Delta\theta) \approx L(\theta) + \nabla_\theta L(\theta)^\top \Delta\theta + \frac{1}{2} \Delta\theta^\top H \Delta\theta, \quad (25)$$

where $H = \nabla_\theta^2 L(\theta)$ denotes the Hessian of the loss and higher-order terms are neglected.

Substituting $\Delta\theta = -\eta\hat{g}$ and writing $g = \nabla L(\theta)$ yields

$$\Delta L \approx -\eta g^\top \hat{g} + \frac{\eta^2}{2} \hat{g}^\top H \hat{g}. \quad (26)$$

Taking the expectation with respect to the stochastic gradient estimator gives

$$\mathbb{E}[\Delta L] = -\eta g^\top g + \frac{\eta^2}{2} \text{tr}(H\Sigma), \quad (27)$$

where $\Sigma = \mathbb{E}[(\hat{g} - g)(\hat{g} - g)^\top]$ is the covariance matrix of the gradient estimator.

This expression shows that the effect of stochastic gradient noise on learning is governed by the interaction between the covariance matrix Σ and the curvature of the loss captured by the Hessian H . In contrast to the first-order analysis of directional alignment, where we saw that cosine similarity affect the expected change in loss directly, the second-order term reveals that estimator variance affects learning through the quantity $\text{tr}(H\Sigma)$. Consequently, variance is particularly harmful when it lies in directions of high curvature in the loss landscape.

Analytical covariance of node perturbation

To understand how estimator variance behaves under node perturbation, we derive an analytical expression for the covariance of the update. As shown in the previous section, the effect of stochastic gradient noise on the expected loss is governed by the quantity $\text{tr}(H\Sigma)$. Deriving the covariance matrix therefore reveals which components of the learning rule contribute to estimator variance.

Following the analysis of [hiratani2022stability], and using the small-perturbation update in Eq. (20), the covariance of the node perturbation update between layers k and l can be written as

$$C_{kl}^{\text{NP}} \approx 2C_{kl}^{\text{SGD}} + \delta_{kl} \left\langle \left(\sum_m \|g_m\|^2 \right) I_k \otimes x_{k-1} x_{k-1}^\top \right\rangle. \quad (28)$$

Here C_{kl}^{NP} denotes the covariance between the updates of layers k and l . Taken together across all layers, these blocks form the covariance matrix of the node perturbation gradient estimator. This matrix corresponds to the covariance Σ appearing in Eq. (18), which determines how estimator variance affects the expected change in loss. The covariance is written in block form here because this representation makes the layer-wise structure of the update easier to analyze. The term C_{kl}^{SGD} denotes the corresponding covariance term for SGD. The Kronecker delta δ_{kl} ensures that the second term contributes only to the variance of layer k . The quantity $\sum_m \|g_m\|^2$ represents the total squared gradient magnitude across layers, and $x_{k-1} x_{k-1}^\top$ denotes the outer product of the presynaptic activity vector. As before, $\langle \cdot \rangle$ denotes expectation over both the perturbation noise and the data distribution.

Several structural observations from Eq. (28) are relevant when considering the variance of the node perturbation estimator. First, the cross-layer covariance is approximately twice that of SGD through the term $2C_{kl}^{\text{SGD}}$, indicating that node perturbation introduces only a moderate increase in covariance between updates of different layers. The dominant additional contribution instead appears in the second term, which affects the variance within each layer through the factor δ_{kl} . Second, this variance term depends on two quantities: the total gradient magnitude $\sum_m \|g_m\|^2$ and the outer product $x_{k-1} x_{k-1}^\top$ of the presynaptic activity. The outer product captures the correlation structure of the inputs to the layer. When the components of x_{k-1} are correlated, this matrix contains large off-diagonal elements, which increases the covariance of the update. In contrast, if the inputs are approximately independent, the contribution primarily appears as diagonal variance. The magnitude of this variance is further scaled by the total gradient energy across all layers, implying that large gradients in any part of the network increase the variance of the node perturbation update throughout the network. Finally, the relevance of this covariance structure must be interpreted in the context of the loss dynamics. As shown earlier, stochastic gradient noise affects the expected loss through the term $\text{tr}(H\Sigma)$, which couples the covariance of

the estimator with the curvature of the loss landscape. Consequently, reducing overall variance is not necessarily the most important objective; variance is most harmful when it lies in directions of high curvature. In contrast to the cosine similarity analysis, where improving directional alignment directly improves the expected loss decrease, variance reduction may therefore be most effective when it specifically targets the components of the covariance structure that interact strongly with the Hessian.

The covariance expression in Eq. (28) relies on several assumptions introduced in the analysis of [hiratani2022stability]. First, as in the previous subsection, the derivation assumes the small-perturbation regime $\sigma \rightarrow 0$, which allows the loss difference to be linearized using a first-order Taylor expansion and leads to the simplified node perturbation update in Eq. (20). Second, the perturbations applied to different neurons are assumed to be independent and identically distributed, which allows expectations of products of perturbation variables to be simplified when computing the covariance of the update. Finally, the full covariance expression contains an additional term involving $\langle g_k x_{k-1}^\top \rangle$. In the analysis of [hiratani2022stability], this term is assumed to be approximately zero, allowing the covariance to be written in the simplified form shown in Eq. (28). Although these assumptions simplify the derivation, the resulting expression still captures the dominant structural contributions to the variance of the node perturbation estimator and therefore provides a useful approximation for analyzing how estimator variance depends on properties of the network and the learning rule.

3.4.3 Stability of the Learning Dynamics

Having examined the directional alignment and variance of the gradient estimator, we now turn to the third quantity that governs the effectiveness of perturbation-based learning: the stability of the resulting learning dynamics. While cosine similarity and estimator variance describe the quality of individual updates, stability determines whether the sequence of updates leads to well-behaved training over time. Following the analysis of [hiratani2022stability], stability can be characterized by tracking the evolution of two macroscopic quantities: the prediction error and the norm of the network weights. Let

$$e(t)$$

denote the prediction error of the student network with respect to the teacher, and let

$$a(t) = \|W\|$$

denote the norm of the network weights.

These two quantities capture whether the learning process remains stable during training. If the prediction error grows without bound, the model fails to approximate the target function. Similarly, if the weight norm increases indefinitely, the parameters leave the regime in which the learning dynamics remain well behaved, which can eventually lead to degraded performance and unstable optimization. Analyzing the joint evolution of the prediction error and the weight norm therefore provides a natural way to study the stability of perturbation-based learning. In the following sections we analyze analytical expressions for the dynamics of these quantities derived in [hiratani2022stability], and discuss how they reveal a fundamental instability mechanism in node perturbation learning.

Analytical Dynamics of Error and Weight Norm

To analyze the stability of node perturbation learning, we examine analytical expressions for the dynamics of the prediction error and the weight norm derived in [hiratani2022stability]. In contrast to the previous metrics, where we first analyzed how cosine similarity and estimator variance influence the expected change in the loss before deriving analytical expressions for these quantities, the stability analysis directly derives dynamical equations for the error and weight norm themselves. Since the prediction error is one of the state variables of this system, these equations describe the evolution of the loss directly, making a separate derivation of the expected loss change unnecessary. The resulting equations therefore provide a dynamical systems description of the learning process.

These equations are derived in an idealized teacher–student setting with deep linear networks. In this setup, a teacher network generates the target outputs for a given input, while a student network is trained to learn the same input–output mapping, corresponding to a standard supervised learning problem. The linear structure allows the high-dimensional learning dynamics to be reduced to a small set of macroscopic variables, which makes the analysis mathematically tractable. While this represents a simplification relative to the nonlinear neural networks considered in this thesis, the empirical results presented in [hiratani2022stability] suggest that the same qualitative behavior also appears in nonlinear networks, a point that will be discussed further in the assumptions section below. In the following sections we first examine the resulting dynamical system in the noise-free case, before analyzing how the presence of teacher noise changes the behavior of the system.

Noise-free case. We first consider the idealized case in which the teacher is noise-free, meaning that the targets provided to the student network are generated deterministically by the teacher without any additional stochastic noise. In this setting, the analysis of [hiratani2022stability] shows that the learning dynamics of node perturbation can be reduced to a system of differential equations describing the joint evolution of the weight norm and the prediction error. Denoting the weight norm by $a(t)$ and the prediction error by $e(t)$, where $e(t)$ represents the mean squared prediction error between the student and teacher outputs, the dynamics can be written as, the dynamics can be written as

$$\dot{a} = c_0 e, \quad (29)$$

$$\dot{e} = -(2 - c_0 a) a e. \quad (30)$$

Here $a(t)$ denotes the norm of the network weights and $e(t)$ denotes the prediction error with respect to the teacher. The quantities $\dot{a}$ and $\dot{e}$ denote the time derivatives of these variables, describing how the weight norm and the prediction error evolve during training. The constant $c_0 > 0$ collects the dependence on the learning rate and network architecture, and in the formulation of [hiratani2022stability] is given by

$$c_0 = \eta L_x L_y (1 + \beta_0), \quad (31)$$

where η is the learning rate, L_x and L_y denote the input and output dimensionality of the linear teacher–student model, and β_0 is a constant determined by the statistics of the perturbations. The coupled equations in Eqs. (29)–(30) therefore describe the joint dynamics of the weight norm and the prediction error during learning.

The dynamical system derived above directly reveals the stability properties of the learning dynamics in the noise-free case. From Eq. (30), the prediction error evolves according to

$$\dot{e} = -(2 - c_0 a) a e. \quad (32)$$

Since $a > 0$ and $e > 0$, the error decreases as long as

$$2 - c_0 a > 0. \quad (33)$$

In this regime the error is reduced while the weight norm increases according to Eq. (29). If the learning rate is sufficiently small, the error converges to zero before the weight norm reaches the critical threshold $a = 2/c_0$. Once $e = 0$, both $\dot{e}$ and $\dot{a}$ vanish and the dynamics become stationary, meaning that the system can remain stable in the noise-free setting.

Noisy teacher case. We next consider the case in which the teacher outputs contain additive noise. In practice this situation is almost unavoidable, since targets obtained from real systems are typically affected by measurement noise, sensor uncertainty, or other sources of stochastic variability. In this setting, the targets provided to the student network are corrupted by a stochastic noise term with variance σ_t^2 . Following the analysis of [hiratani2022stability], the resulting learning dynamics become

$$\dot{a} = c_0 \left(e + \frac{\sigma_t^2}{L_x} \right), \quad (34)$$

$$\dot{e} = -2ae + c_0 a^2 \left(e + \frac{\sigma_t^2}{L_x} \right), \quad (35)$$

where σ_t^2 denotes the variance of the teacher noise and L_x is the input dimensionality of the linear teacher–student model. Relative to the noise-free case, the additional term σ_t^2/L_x reflects the irreducible error introduced by noisy targets, which modifies the evolution of both the weight norm and the prediction error.

The presence of teacher noise fundamentally changes the stability properties of the system. In this case the error dynamics become

$$\dot{e} = -2ae + c_0 a^2 \left(e + \frac{\sigma_t^2}{L_x} \right). \quad (36)$$

Rearranging this expression shows that the error decreases only when

$$e > \frac{c_0 a \sigma_t^2}{L_x(2 - c_0 a)}. \quad (37)$$

The analysis above reveals a fundamental limitation of node perturbation learning: in the presence of teacher noise the learning dynamics are ultimately unstable. Once the prediction error falls below the threshold derived above, the sign of $\dot{e}$ becomes positive and the error begins to increase again. Because teacher noise prevents the error from converging to zero, the weight norm continues to grow according to Eq. (34), eventually forcing the system across this instability threshold. The dynamical system therefore predicts that, when the targets contain noise, instability is unavoidable in the long run. This behavior is clearly undesirable for a learning algorithm, since it implies that the parameters will eventually leave the regime in which the model approximates the target function well.

While this result may initially appear to severely limit the usefulness of node perturbation, two observations mitigate this concern. First, [hiratani2022stability] demonstrate that applying weight normalization prevents the unbounded growth of the weight norm and stabilizes the learning dynamics in practice. This modification does introduce bias, since constraining the weights restricts the space of solutions available to the network, but it nevertheless provides a practical way of avoiding divergence. Second, the instability predicted by the dynamical system may occur only after long training times. In particular, the threshold depends on the constant c_0 , which is determined by the learning rate and the network size, suggesting that appropriate choices of these parameters may delay the onset of instability. In practice it is therefore possible that useful learning occurs well before the instability becomes visible. These observations suggest that although node perturbation exhibits an inherent instability mechanism, the practical consequences depend strongly on the training regime and warrant further investigation.

The stability analysis described above relies on several simplifying assumptions introduced in [hiratani2022stability]. Most importantly, the dynamical system in Eqs. (29)–(35) is derived in a teacher–student setting with deep linear networks, meaning that both the teacher and student networks consist only of linear transformations without nonlinear activation functions. This assumption allows the high-dimensional parameter dynamics of the network to be reduced to the two macroscopic variables $a(t)$, representing the weight norm, and $e(t)$, representing the squared prediction error. While this represents a simplification relative to the nonlinear neural networks considered in this thesis, the authors demonstrate empirically that the same qualitative behavior predicted by the linear analysis also appears in nonlinear networks trained with node perturbation, including the steady growth of the weight norm and the eventual instability in the presence of teacher noise. In addition, the derivation assumes a large-width regime in which sums over weights can be approximated by expectations over Gaussian random variables, consistent with Xavier–Glorot style initialization, which simplifies the analysis by allowing correlations between individual parameters to be neglected. Finally, the dynamics are analyzed in the continuous-time limit obtained by considering sufficiently small learning rates so that the discrete weight updates can be approximated by differential equations, and the gradient estimator itself is based on the same first-order small-perturbation approximation used earlier when deriving the cosine similarity and covariance expressions. Although these assumptions simplify the analysis considerably, together they allow the derivation to reveal the mechanism responsible for instability in perturbation-based

learning rules while still providing insight into the behavior observed in more realistic nonlinear networks.

Diffusion interpretation. Equation (??) describes a discrete-time Ornstein–Uhlenbeck process. If curvature provides insufficient contraction, the dynamics reduce to a random walk:

$$\theta_{t+1} \approx \theta_t - \eta z_t, \quad (38)$$

whose variance grows linearly in time. Even when the mean remains near θ^* , the variance — and thus the expected loss — can increase.

3.5 Theory-Driven Extensions to INP

3.5.1 TO BE REMOVED (most likely): Geometry Aware Perturbation

Exploration Anisotropy in Large Networks

Perturbation-based learning methods estimate gradients by injecting noise into the parameters and observing the resulting change in loss. In its simplest form, weight perturbation samples noise isotropically in parameter space,

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (39)$$

and forms a gradient estimate from the scalar loss difference.

However, the mapping from parameters to loss is highly anisotropic in large neural networks. Some parameters induce large changes in model output and loss when perturbed, while others have negligible effect. As a result, isotropic noise in parameter space produces highly anisotropic changes in function space. A small subset of parameters can dominate the observed loss variation, while many perturbations contribute little information. This leads to inefficient exploration and poor signal-to-noise ratio in the scalar learning signal.

This phenomenon is closely related to classical observations in high-dimensional optimization, where parameter-space isotropy does not imply function-space isotropy (Amari, 1998; Martens, 2014). In particular, natural gradient methods rescale updates according to the Fisher information matrix in order to ensure that steps of equal magnitude correspond to approximately equal changes in model behavior (Amari, 1998).

While natural gradient rescales *updates*, perturbation learning offers an alternative intervention point: rescaling the *noise distribution itself*. Instead of sampling perturbations isotropically in parameter space, we aim to sample perturbations approximately isotropically in loss space.

Objective: Isotropic Perturbations in Loss Space

Let θ_i denote a parameter and let ΔL_i denote the change in loss induced by perturbing only θ_i . The goal of geometry-aware perturbations is to enforce

$$\mathbb{E}[(\Delta L_i)^2] \approx \text{constant across } i, \quad (40)$$

so that each perturbation induces comparable expected impact on the loss.

Under a first-order approximation,

$$\Delta L_i \approx \frac{\partial L}{\partial \theta_i} \epsilon_i, \quad (41)$$

so isotropy in loss space suggests sampling noise as

$$\epsilon_i \sim \mathcal{N}\left(0, \frac{\sigma^2}{s_i^2 + \delta}\right), \quad (42)$$

where s_i is an estimate of sensitivity, ideally proportional to

$$s_i^2 \approx \left(\frac{\partial L}{\partial \theta_i}\right)^2. \quad (43)$$

To ensure fair comparison with isotropic perturbations, total injected noise energy is normalized such that

$$\sum_i \text{Var}(\epsilon_i) = \text{constant}. \quad (44)$$

The central hypothesis is that exploration inefficiency due to anisotropy may limit perturbation learning, and that geometry-aware noise sampling can improve learning dynamics.

Experimental Evaluation

We evaluate the hypothesis through four experiments, ordered from theoretical upper bound to practical approximations.

Experiment 1: Oracle Sensitivity Scaling

This experiment tests whether geometry-aware noise improves performance in principle.

Using backpropagation, we compute the true gradient magnitude

$$s_i = \left| \frac{\partial L}{\partial \theta_i} \right|. \quad (45)$$

Noise is then sampled as

$$\epsilon_i \sim \mathcal{N}\left(0, \frac{\sigma^2}{s_i^2 + \delta}\right), \quad (46)$$

with total variance normalized.

If oracle scaling improves convergence speed or final performance relative to isotropic perturbations, then exploration anisotropy is likely a limiting factor. If not, then anisotropy is unlikely to be the dominant bottleneck.

Experiment 2: Eligibility-Based Local Approximation

Since true gradients are unavailable in biologically plausible settings, we approximate sensitivity using local eligibility traces:

$$e_{ij} = x_i f'(z_j), \quad (47)$$

where x_i is presynaptic activity and $f'(z_j)$ is the derivative of the postsynaptic activation.

We maintain a running average

$$\hat{s}_{ij}^2 = \text{EMA}(e_{ij}^2), \quad (48)$$

and sample perturbations as

$$\epsilon_{ij} \sim \mathcal{N}\left(0, \frac{\sigma^2}{\hat{s}_{ij}^2 + \delta}\right). \quad (49)$$

This provides a biologically local approximation to Fisher-diagonal scaling.

Experiment 3: Layer-Wise Sensitivity Scaling

An intermediate approximation is to scale perturbations at the layer level.

For layer k , we estimate average gradient magnitude

$$s_k^2 = \text{EMA}\left(\frac{1}{|\theta_k|} \sum_{i \in k} \hat{g}_i^2\right), \quad (50)$$

and scale perturbations within that layer as

$$\epsilon_i \sim \mathcal{N}\left(0, \frac{\sigma^2}{s_k^2 + \delta}\right). \quad (51)$$

This tests whether coarse sensitivity equalization across depth improves exploration.

Experiment 4: Adam-Inspired Sensitivity Scaling

Finally, we consider an adaptive estimate inspired by RMSProp/Adam (Kingma & Ba, 2015).

We maintain a running second moment estimate

$$v_i = \text{EMA}(\hat{g}_i^2), \quad (52)$$

where $\hat{g}_i$ is the perturbation-based gradient estimate.

Noise is sampled as

$$\epsilon_i \sim \mathcal{N}\left(0, \frac{\sigma^2}{\sqrt{v_i} + \delta}\right). \quad (53)$$

This approach uses second-order statistics of the learning signal itself as a proxy for functional sensitivity.

4 Case Study

4.1 Experimental Setup

4.2 Learning Problems and Datasets

- Noisy Sinus Regression
- California Housing Regression
- Fashion MNIST
- Emnist Balanced

4.3 Algorithms

4.4 Results

5 Concluding Remarks

5.1 Discussion

5.2 Future Work

5.3 Conclusion

Appendix

Table 1: Comparison of perturbation noise sampling in the literature.

Reference	Method(s)	Perturbed variable	Noise sampling	Notes
[hiratani2022stability]	Node perturbation; weight-normalized node perturbation	Pre-activation	$\sigma \xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	Weight-normalized version renormalizes incoming weights.
[dalm2024effective]	Node perturbation; iterative node perturbation; activity-based node perturbation	Pre-activation	$\xi_l \sim \mathcal{N}(0, \sigma^2 I_l)$	Decorrelated versions exist, they use the same distribution.
[flower1992summed]	Neuron perturbation; weight perturbation; summed weight neuron perturbation	Weight / pre-activation	$\epsilon_i, \epsilon_{ij} \in \{-\sigma, \sigma\}$	Random-polarity finite-difference perturbations.
[widrow1990thirty]	Madaline Rule III	Pre-activation	Small perturbation ϵ_i ; no distribution specified	Early finite-difference neuron perturbation.
[cauwenberghs1993fast]	Stochastic error descent	Weights	$\mathbb{E}[\epsilon_i \epsilon_j] = \sigma^2 \delta_{ij}$	Any zero-mean, uncorrelated perturbations with equal variance; simulations use $\epsilon_i = \sigma z_i$, $z_i \sim \text{Unif}\{-1, 1\}$.
[werfel2005learning]	Node perturbation; weight perturbation	Output / weight	NP: $\epsilon \sim \mathcal{N}(0, N \sigma^2 I_M)$; WP: $\epsilon \sim \mathcal{N}(0, \sigma^2 I_{MN})$	Node noise scaled by $\sqrt{N}$ to match the output noise induced by weight perturbation.
[zuge2023weight]	Weight perturbation; node perturbation	Weight / pre-activation	WP: $\epsilon_{ij}^{\text{WP}} \sim \mathcal{N}\left(0, \frac{\sigma_{\text{eff}}^2}{\alpha^2 N_{\text{eff}}}\right)$; NP: $\epsilon_{it}^{\text{NP}} \sim \mathcal{N}(0, \sigma_{\text{eff}}^2)$	WP variance matched to NP output variance. N_{eff} : effective input dimension. α : error convergence factor.
[cho2011node]	Node perturbation; noisy-baseline node perturbation	Output	$\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$, $\zeta_i \sim \mathcal{N}(0, \sigma_\zeta^2)$	Second perturbation replaces noiseless baseline; no layer-size scaling.
[hara2017statistical]	Single node perturbation; double node perturbation	Output	$\epsilon_k \sim \mathcal{N}(0, \sigma_\epsilon^2)$, $\zeta_k \sim \mathcal{N}(0, \sigma_\zeta^2)$	Statistical-mechanics analysis of noisy-baseline NP; no layer-size scaling.

Table 2: Comparison of perturbation-based update rules in the literature.

Reference	Method	Perturbed variable	Noise sampling	Update rule
[hiratani2022stabilizing]	Node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
[hiratani2022stabilizing]	Weight-normalized node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top,$ $w_i^k \leftarrow \frac{\ w_i^k\ }{\ w_i^k - \eta w u_i^k\ } (w_i^k - \eta w u_i^k)$
[dalm2024effective]	Node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
[dalm2024effective]	Iterative node perturbation	Pre-activation	$v_k \sim \mathcal{N}(0, I_k),$ perturbation $h v_k$	$U_k = \sqrt{N_k} \Delta \ell \frac{v_k}{\ v\ _2^2} x_{k-1}^\top$
[dalm2024effective]	Activity-based node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta a_k}{\ \delta a\ _2^2} x_{k-1}^\top$
[dalm2024effective]	Decorrelated node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k (x_{k-1}^*)^\top$
[dalm2024effective]	Decorrelated iterative node perturbation	Pre-activation	$v_k \sim \mathcal{N}(0, I_k),$ perturbation $h v_k$	$U_k = \sqrt{N_k} \Delta \ell \frac{v_k}{\ v\ _2^2} (x_{k-1}^*)^\top$
[dalm2024effective]	Decorrelated activity-based node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta a_k}{\ \delta a\ _2^2} (x_{k-1}^*)^\top$
[flower1992summed]	Neuron perturbation	Pre-activation	$\epsilon_i = \sigma z_i, z_i \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\epsilon_i} x_j$
[flower1992summed]	Weight perturbation	Weight	$\epsilon_{ij} = \sigma z_{ij}, z_{ij} \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\epsilon_{ij}}$
[flower1992summed]	Summed weight neuron perturbation	Incoming weights	$\epsilon_{ij} = \sigma z_{ij}, z_{ij} \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\epsilon_{ij}}, \quad r_i = \sum_m \epsilon_{im} x_m$
[widrow1990thirty]	Madaline Rule III	Pre-activation	Small perturbation ϵ_i ; no distribution specified	$U_{ij} = \frac{\Delta \ell}{\epsilon_i} x_j$
[cauwenberghs1993stochastic]	Stochastic error descent	Weights	$\mathbb{E}[\epsilon_i \epsilon_j] = \sigma^2 \delta_{ij}$	$U = \Delta \ell \epsilon$
[werfel2005learning]	Node perturbation	Output	$\epsilon \sim \mathcal{N}(0, N \sigma^2 I_M)$	$U = \frac{\Delta \ell}{N \sigma^2} \epsilon \mathbf{x}^\top$
[werfel2005learning]	Weight perturbation	Weight	$\epsilon \sim \mathcal{N}(0, \sigma^2 I_{MN})$	$U = \frac{\Delta \ell}{\sigma^2} \epsilon$
[zuge2023weight]	Weight perturbation	Weight	$\epsilon_{ij}^{\text{WP}} \sim \mathcal{N}\left(0, \frac{\sigma_{\text{eff}}^2}{\alpha^2 N_{\text{eff}}}\right)$	$U_{ij} = \frac{\Delta L}{\sigma_{\text{WP}}^2} \epsilon_{ij}^{\text{WP}}, \quad \sigma_{\text{WP}}^2 = \frac{\sigma_{\text{eff}}^2}{\alpha^2 N_{\text{eff}}}$
[zuge2023weight]	Node perturbation	Pre-activation	$\epsilon_{it}^{\text{NP}} \sim \mathcal{N}(0, \sigma_{\text{eff}}^2)$	$U_{ij} = \frac{\Delta L}{\sigma_{\text{eff}}^2} \sum_{t=1}^T \epsilon_{it}^{\text{NP}} x_{jt}$
[cho2011node]	Node perturbation	Output	$\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$	$U_{ij} = \Delta L \epsilon_i x_j$
[cho2011node]	Noisy-baseline node perturbation	Output	$\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2),$ $\zeta_i \sim \mathcal{N}(0, \sigma_\zeta^2)$	$U_{ij} = \Delta L \epsilon_i x_j, \quad \Delta L = L_\epsilon - L_\zeta$
[hara2017statistical]	Single node perturbation	Output	$\epsilon_k \sim \mathcal{N}(0, \sigma_\epsilon^2)$	$U_{jk} = \frac{\Delta L}{N \sigma_\epsilon^2} \epsilon_k x_j, \quad \Delta L = L_\epsilon - L$
[hara2017statistical]	Double node perturbation	Output	$\epsilon_k \sim \mathcal{N}(0, \sigma_\epsilon^2),$ $\zeta_k \sim \mathcal{N}(0, \sigma_\zeta^2)$	$U_{jk} = \frac{\Delta L}{N \sigma_\epsilon^2} \epsilon_k x_j, \quad \Delta L = L_\epsilon - L_\zeta$